

Chapter 15: Minors and Algorithms

15.1 Minors and Well-quasi-orderings

A **quasi-ordering** is a reflexive transitive relation. A set X is **well-quasi-ordered** (WQO) by quasi-ordering $\leq$, if given any infinite subsequence in X there must exist two elements x_i, x_j with $i < j$ such that $x_i \leq x_j$. It can readily be shown that:

FACT. *A set is WQO iff it contains neither an infinite antichain (no two elements comparable) nor an infinite strictly decreasing sequence.*

A graph G is a **topological minor** of graph H if H contains a subgraph that is a subdivision of G . People also say that G is a homeomorphic subgraph of H .

THEOREM. *[Kruskal] The finite trees are WQO by the topological minor relation.*

That is, given any infinite sequence of trees, there is $i < j$ such that tree T_j contains a subdivision of tree T_i as a subgraph.

THEOREM. *[Robertson & Seymour] The finite graphs are WQO by the minor relation.*

As an immediate corollary we get:

THEOREM. *Every graph property closed under minors has a characterization by a finite list of forbidden minors.*

In particular, for every surface there is a finite list of forbidden minors. Actually, they proved this consequence as a stepping stone to the overall theorem.

For example, a **linkless embedding** of a graph is an embedding of the graph in 3 dimensions such that there are no two interlocking cycles. Having a linkless embedding is clearly closed under taking minors. So before the problem was not known to be decidable; now it is known to be in P, since there is a polynomial time algorithm to test for the existence of a fixed minor. But note that the algorithm has a huge hidden constant, and is nonconstructive.

15.2 Treewidth

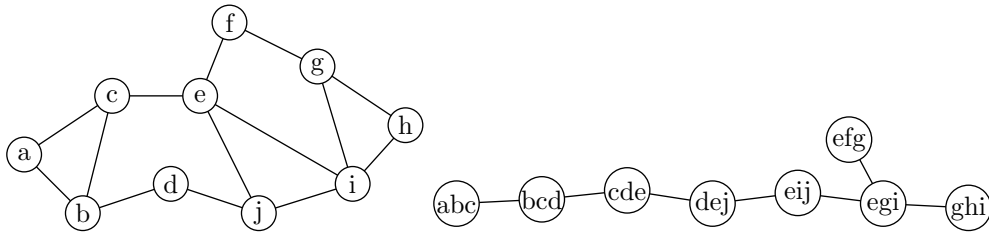
A **tree decomposition** of a graph G is a tree T and a mapping from the vertex set of T to the induced subgraphs of G such that:

- 1) every vertex of G is in V_t for some $t \in T$;
- 2) every edge of G has both ends in V_t for some $t \in T$;
- 3) if vertex t_2 lies on the t_1 - t_3 path in T , then $V_{t_1} \cap V_{t_3} \subseteq V_{t_2}$.

We call the sets V_t the **pieces** of G .

The width of a decomposition is 1 less than the maximum number of vertices in V_t . The **treewidth** of a graph is the minimum width of a tree decomposition.

Example 1. Here is a graph G and a tree decomposition T of width 2:



Example 2. There is always a trivial tree decomposition where the tree T has one vertex and its piece is the whole of G .

Example 3. If G is a tree, then T is the subdivision of G : If vertex t of T is a vertex of G , then V_t is just the vertex; if vertex t of T is an edge of G , then V_t is the edge and its ends.

Example 4. A cycle has treewidth 2. So a connected graph has treewidth 1 if and only if it is a tree.

A graph property is called **hereditary** if it is closed under taking subgraphs. Note that in the natural way, having treewidth at most k is hereditary. The graphs of treewidth k are also known as the **partial k -trees**, meaning subgraphs of k -trees. A **k -tree** is built by starting with K_{k+1} and at each stage adding a vertex and joining it to a k -clique. The graphs of treewidth at most 2 are also known as the **series-parallel** graphs, which are characterized as those graphs without K_4 as a minor.

THEOREM. (Robertson & Seymour) The graphs of treewidth at most k are WQO by minors.

Example 5. The $n \times n$ grid G_n has treewidth at least n .

THEOREM. (Robertson & Seymour) A graph of sufficiently large treewidth contains either K_m or the grid G_m as a minor.

15.3 Postorder Traversal Tree Algorithms

Many graph parameters can be calculated on trees by a postorder traversal algorithm. In such an algorithm, one picks any vertex as root and then processes all vertices such that each vertex is processed after its children. At each vertex a tuple of values is calculated using the values for the children. The overall algorithm runs in linear time provided processing a vertex runs in time proportional to the number of children.

The tuple of values calculated at a vertex v are usually the values of modified versions of the parameter applied to the subtree rooted at v , where the criterion is either relaxed or strengthened at vertex v . These criteria are determined by what states are possible if one restricts a valid set to that subtree. For example, in domination the states are that the root is in the set, the root is not in the set but dominated, or the root is not dominated.

15.3.1 Example: Independent Domination

For an example: consider the independent domination number of a tree. If we consider an independent dominating set S restricted to a subtree, there are three possibilities for the root of the subtree:

- (1) it is in S ,
- (2) it is not in S but dominated by one of its children,
- (3) it is not dominated within that subtree.

So let $f_i(T_v)$ be the minimum cardinality of a set of vertices in T_v such that the set S can be extended to an independent dominating set in the whole graph—that is, the set S is independent and dominates at least $V(T_v) - \{v\}$ —and the root v has property (i).

One can calculate these parameters as follows. If v is going to be in, then each of its children must be in state 2 or 3. Thus,

$$f_1(T_v) = 1 + \sum_c \min(f_2(T_c), f_3(T_c)).$$

If v is not in, then none of its children can be in state 3. Further, if v is not dominated, then none of its children can be in state 1. Thus

$$f_3(T_v) = \sum_c f_2(T_c).$$

If v is dominated, then at least one of its children is in state 1. So $f_2(T_v)$ is the minimum sum of $f_1(T_c)$ or $f_2(T_c)$ over its children, subject to the constraint that at least one of these must be f_1 . This evaluates to:

$$f_2(T_v) = \begin{cases} \sum_c \min(f_1(T_c), f_2(T_c)) & \text{if } f_1(T_c) \leq f_2(T_c) \text{ for some } c, \\ \sum_c f_2(T_c) + \min_c f_1(T_c) - f_2(T_c) & \text{otherwise.} \end{cases}$$

Finally, having done the traversal, one can calculate the actual independent domination number of the tree as $i(T) = \min(f_1(T_r), f_2(T_r))$ where r is the overall root.

Furthermore, to determine the actual minimum independent dominating set, one can work back down the tree, looking at each stage at which of the f_i was taken at the parent and calculating whether the vertex is in or out.

15.4 Treewidth and Algorithms

One can extend the postorder traversal idea to graphs with bounded treewidth. Given a tree decomposition T , we root the tree arbitrarily. Then for a vertex $t \in T$, define G_t as the subgraph of G induced by the pieces corresponding to t and its descendants.

Note that if t_1 and t_2 are children of t in T , then graphs $G_{t_1} - V_t$ and $G_{t_2} - V_t$ are disjoint and not joined by an edge. This follows by the property of tree decomposition.

Now, to get a dynamic programming algorithm, do a postorder traversal on T . If we remove one piece, then we split the graph into components and the only interaction is with the piece. So we calculate, for each vertex t of T and every possible interaction with V_t , the optimum of the parameter for G_t given its behavior on V_t . This is calculated from such values for its children. The algorithm takes time linear in the number of vertices of T , but exponential or worse in the treewidth.

For example: suppose we want to calculate the domination number. Then for G_t we calculate for every possible $R, S \subseteq V_t$ the minimum number of vertices in a set D that dominates at least $V(G_t) - R$, such that $D \cap V_t = S$.